

ICML 2023: Making Tensor Rematerialization Scale by Making the Problem Smaller

Moccasin, from Qualcomm AI Research and USC, recasts rematerialization as a constraint program with a linear rather than quadratic number of decision variables

On the edge hardware that runs today's neural networks (phones, wearables, on-device accelerators), the first wall you hit is usually not compute but memory. On-chip memory is small, and the moment a model's intermediate tensors no longer fit, the system has to reach for much slower external memory, which drags down both inference and training.

A well-known way around this is tensor rematerialization: instead of keeping every intermediate tensor in memory, you recompute some of them on demand, trading extra compute for a lower peak memory footprint. The catch is that recomputation costs time. That turns the problem into a scheduling one: given a compute graph and a fixed memory budget, which tensors should you retain and which should you recompute so that total execution time is as small as possible while peak memory never exceeds the budget? In general this is a PSPACE-complete combinatorial optimization.

The leading prior method, Checkmate, casts this as a mixed-integer linear program (MILP). It is expressive, but its number of Boolean decision variables grows with the square of the node count, $O(n^2)$. Past a few hundred nodes the solver either times out or runs out of memory. Moccasin, the method introduced in this paper, changes the formulation rather than the solver: it describes each node's tensor lifetime with a handful of retention intervals, which cuts the count of discrete variables from quadratic to linear, $O(n)$. On graphs with hundreds of nodes and thousands of edges, that makes it up to roughly an order of magnitude faster than Checkmate.

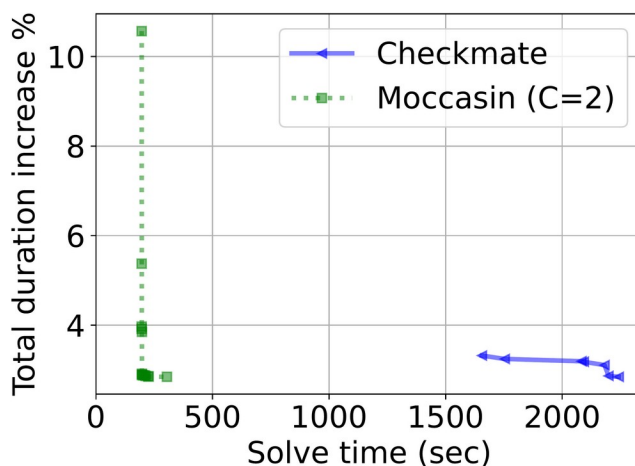


Figure 1. Total duration increase versus solve time for both methods on a real-world compute graph with $n = 442$ nodes and $m = 1247$ edges, with the memory budget set to 80% of the peak memory of the

rematerialization-free schedule. Moccasin (green) converges to a low-overhead solution within a few hundred seconds; Checkmate (blue) reaches comparable quality only around 2000 seconds.

Paper: <https://arxiv.org/abs/2304.14463>

Why rematerialization is hard to solve at scale

Every node in a compute graph is one operator that produces an intermediate tensor. Keeping a tensor around saves the time you would otherwise spend recomputing it but costs memory; recomputing it saves memory but costs time. The memory budget is what turns this trade-off into a constrained optimization. Checkmate handles it by attaching a Boolean variable to the question "is this tensor resident in memory at this time step?" for every tensor and every step, which is where the $O(n^2)$ variables come from. The formulation works well on small and medium graphs, but the variable count explodes once the graph reaches a few hundred nodes, at which point the solver runs out of either time or memory.

The core idea: retention intervals

Moccasin's key observation is that over an entire execution, a tensor is actually resident in memory during only a few stretches of time. Rather than track a separate Boolean for every time step, describe those stretches directly as intervals.

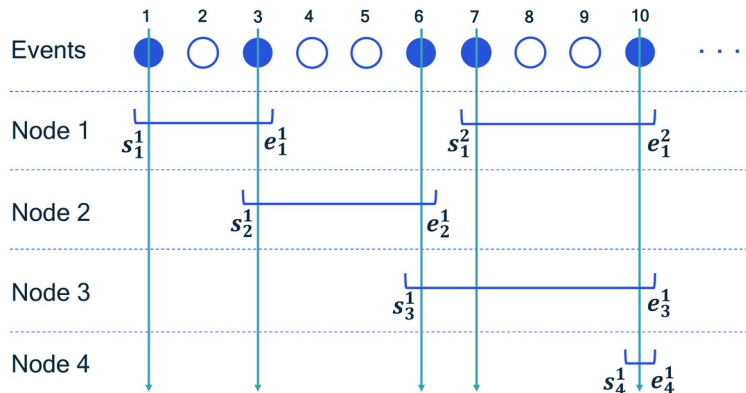


Figure 2. The retention-intervals formulation. The horizontal axis is time partitioned into discrete events, and each row is one node. An interval runs from a start event s to an end event e : its first event marks recomputing the node and writing its tensor to memory, and its end marks the tensor being freed. Node 1 uses two intervals (it is recomputed once); the others use a single interval.

Each node is allowed up to C_v intervals, and each interval is described by an integer start, an integer end, and a Boolean activity flag. The objective minimizes a weighted sum over the active compute events, with each node's execution time as its weight, which is exactly the extra runtime that recomputation adds. The memory budget is expressed with the constraint-programming cumulative constraint (tensor sizes as resource demands, the budget M as capacity), and the data dependencies between nodes are handled with the reservoir constraint. The whole model is handed to Google OR-Tools' CP-SAT solver. In the experiments the per-node interval cap C_v is fixed at 2, so the total number of integer variables is only $O(C_n)$, linear in the graph size.

Shrinking the search space with stages

Moccasin also offers an optional variant that snaps interval start times onto stages carved out of an input topological ordering.

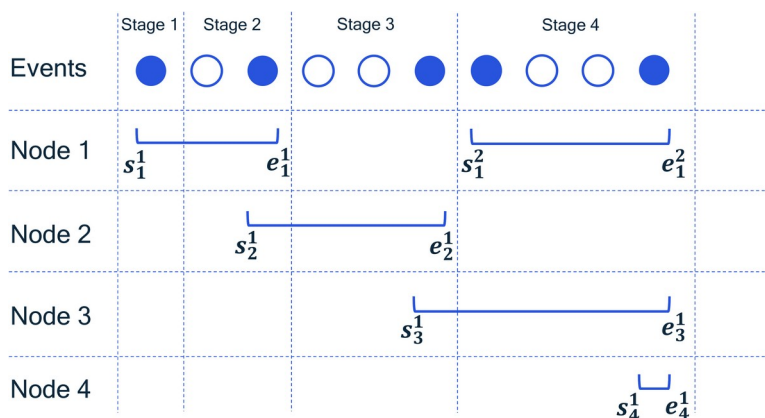


Figure 4. Grouping events into stages. When an input topological ordering is enforced, the start of each node's first interval becomes a fixed value rather than a free variable, which shrinks the overall search space and speeds up the solve.

The paper's ablation shows that this topological-ordering restriction, while it slightly enlarges the variables' domains, substantially compresses the search space and makes the solve faster; and setting C_v as small as 2 costs almost nothing in solution quality.

Results: same quality, far less solve time

The authors compare the two methods on a family of random layered graphs, G1 through G4 (from 100 up to 1000 nodes, and up to 5875 edges), and on real-world compute graphs, with the memory budget set to 80% and 90% of the rematerialization-free peak memory.

Table 1. Solve time on the random layered graphs at a memory budget of 80% of the initial peak memory. "Duration increase" is the extra execution time, as a percentage, that rematerialization adds relative to the no-rematerialization schedule; "—" means the method found no feasible solution within the time limit.

Graph (n, m)	Checkmate time (s)	Moccasin time (s)	Duration increase (Moccasin)
G1 (100, 236)	18.5	9.3	0.8%
G2 (250, 944)	685.1	55.0	0.9%
G3 (500, 2461)	—	1803.3	0.7%
G4 (1000, 5875)	—	3612.9	0.7%

Two things stand out. First, on the graphs both methods can solve (G1 and G2), Moccasin's solution quality (duration increase) is essentially the same as Checkmate's, but it gets there several to more than ten times faster. Second, and more telling: once the graph grows to 500 and 1000 nodes, Checkmate finds no feasible solution within a three-hour limit and exits out of memory, whereas Moccasin still converges to a good solution, under 1% duration increase, in under an hour. The curves on the real-world graph in Figure 1 are the visual version of that order-of-magnitude gap.

Takeaway and boundaries

The lesson in Moccasin is a plain one: reformulating a problem to use fewer variables often changes what is solvable more than swapping in a stronger solver would. By expressing rematerialization decisions as a small set of retention intervals, Moccasin brings the discrete-variable count down from $O(n^2)$ to $O(n)$, which lets it handle compute graphs that prior methods simply could not, up to about an order of magnitude faster, while holding the added runtime overhead under 5%.

It is worth being clear about the boundary. Moccasin optimizes the step that produces an execution sequence, and that step is hardware-agnostic: the resulting sequence can run on any CPU or GPU. But it currently models rematerialization only, and does not yet incorporate paging (temporarily writing intermediate data out to external memory); the authors list combining the two as future work. For anyone who has to plan memory for large models at compile time, Moccasin offers an option that scales further than what came before.

In practice, the payoff is largest exactly where the older approach breaks down. Planning compile-time memory for a large model, or fitting training and inference onto devices with limited on-chip or GPU memory, is precisely the regime of big compute graphs, and the bigger the graph the wider Moccasin's margin; on small graphs the two methods are comparable. What the reformulation really buys, then, is not a uniform speedup on every case but a larger set of problems that can be solved at all.